

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA HỆ THỐNG THÔNG TIN

NGÔ TIẾN SỸ – 23521367
NGUYỄN VĂN NAM – 23520982

BÁO CÁO ĐỒ ÁN
XÂY DỰNG KIẾN TRÚC KHO HỒ DỮ LIỆU THỜI GIAN THỰC CHO HỆ
THỐNG GIAO THÔNG THÔNG MINH TRÊN CỤM KUBERNETES
**Building a Real-Time Data Lakehouse Architecture for Intelligent Transportation
Systems on a Kubernetes Cluster**

CHUYÊN NGÀNH HỆ THỐNG THÔNG TIN

GIẢNG VIÊN HƯỚNG DẪN
THS. NGUYỄN HỒ DUY TRÍ

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA HỆ THỐNG THÔNG TIN

NGÔ TIẾN SỸ – 23521367
NGUYỄN VĂN NAM – 23520982

BÁO CÁO ĐỒ ÁN
XÂY DỰNG KIẾN TRÚC KHO HỒ DỮ LIỆU THỜI GIAN THỰC CHO HỆ
THỐNG GIAO THÔNG THÔNG MINH TRÊN CỤM KUBERNETES
**Building a Real-Time Data Lakehouse Architecture for Intelligent Transportation
Systems on a Kubernetes Cluster**

CHUYÊN NGÀNH HỆ THỐNG THÔNG TIN

GIẢNG VIÊN HƯỚNG DẪN
THS. NGUYỄN HỒ DUY TRÍ

TP. HỒ CHÍ MINH, 2026

LỜI CẢM ƠN

Nhóm thực hiện xin trân trọng cảm ơn ThS. Nguyễn Hồ Duy Trí đã hướng dẫn, góp ý và định hướng trong quá trình xây dựng đề tài. Những trao đổi về kiến trúc dữ liệu, xử lý luồng, triển khai trên Kubernetes và cách diễn giải kết quả thực nghiệm giúp nhóm hoàn thiện báo cáo theo hướng rõ ràng và có khả năng kiểm chứng hơn.

Nhóm cũng xin cảm ơn Trường Đại học Công nghệ Thông tin, Đại học Quốc gia TP. Hồ Chí Minh và Khoa Hệ thống Thông tin đã tạo môi trường học tập để nhóm có thể tiếp cận các kiến thức về cơ sở dữ liệu phân tán, dữ liệu lớn, hạ tầng Kubernetes, GitOps và quan sát hệ thống. Các kiến thức này là nền tảng để nhóm thiết kế, triển khai và đánh giá hiện vật Continux trong phạm vi đồ án.

Do giới hạn thời gian và hạ tầng thực nghiệm, báo cáo khó tránh khỏi thiếu sót. Nhóm mong nhận được góp ý từ giảng viên và các bạn để tiếp tục cải thiện hệ thống trong các phiên bản sau.

TP. Hồ Chí Minh, 2026

Nhóm thực hiện

MỤC LỤC

Tóm tắt báo cáo.....	1
Chương 1. Mở đầu	2
1.1 Đặt vấn đề	2
1.2 Lý do chọn đề tài	2
1.3 Mục tiêu nghiên cứu	2
1.4 Đối tượng và phạm vi	3
1.5 Câu hỏi nghiên cứu	3
1.6 Đóng góp và cấu trúc báo cáo	3
Chương 2. Tổng quan và cơ sở lý thuyết.....	5
2.1 Kho hồ dữ liệu và Apache Iceberg	5
2.2 Xử lý luồng, trạng thái và khung nhìn vật lý	5
2.3 Broker tương thích Kafka và RisingWave	5
2.4 Chuyển giao Blue/Green.....	6
2.5 GitOps và khả năng quan sát	6
2.6 Ursa và nghiên cứu liên quan.....	7
Chương 3. Nghiên cứu thực nghiệm và kiến trúc hệ thống	8
3.1 Phương pháp nghiên cứu	8
3.2 Dữ liệu và cấu hình phát lại	8
3.3 Kiến trúc tổng thể	9
3.4 Cấu trúc triển khai	9
3.5 Đối tượng SQL và chuyển giao Blue/Green	11
3.6 Nguồn bằng chứng và khả năng truy vết.....	12
Chương 4. Kết quả thực nghiệm, đánh giá và bàn luận	13
4.1 Môi trường và phiên bản.....	13
4.2 Trạng thái nền và khả năng lặp lại.....	13
4.3 Dữ liệu đầu vào có thể truy vết	14
4.4 Kết quả phát lại theo bốn cấu hình	15
4.5 Kết quả chuyển giao Blue/Green	17
4.6 Đầu ra Iceberg và trạng thái sau dọn dẹp	17

4.7	Tổng hợp đánh giá	19
4.8	Thảo luận kết quả	19
4.9	Hạn chế	20
Chương 5.	Kết luận	20
Chương 6.	Hướng phát triển.....	22
Tài liệu tham khảo		23
Phụ lục A.	SQL cốt lõi.....	25
A.1	Nguồn sự kiện, bảng tra cứu và MV nền.....	25
A.2	MV Green và thao tác hoán đổi	26
A.3	Đích ghi Iceberg	26
Phụ lục B.	Cấu hình vận hành trích yếu	26
Phụ lục C.	Chỉ mục bằng chứng	27
Phụ lục D.	Danh sách hình minh chứng.....	28

DANH MỤC HÌNH

Hình 3.1:	Kiến trúc, luồng xử lý và quy trình chuyển giao của Continux 2.1.0.....	9
Hình 4.1:	Bảng điều khiển hiệu năng xử lý luồng trong lượt benchmark-high.....	16
Hình 4.2:	Bảng điều khiển tài nguyên trong cửa sổ phát lại của lượt benchmark-high.....	16
Hình 4.3:	Bảng điều khiển chuyển giao sau hoán đổi MV công khai trong lượt benchmark-high.....	18
Hình 4.4:	Bảng điều khiển tính toán vện quan sát được sau chuyển giao trong lượt benchmark-high.	18

DANH MỤC BẢNG

Bảng 1.1:	Câu hỏi nghiên cứu và bằng chứng trả lời	4
Bảng 3.1:	Nguồn dữ liệu và cấu hình phát lại.....	8
Bảng 3.2:	Cấu trúc cụm K3s thực nghiệm	9
Bảng 3.3:	Đối tượng dữ liệu cốt lõi.....	11
Bảng 3.4:	Nhóm tệp bằng chứng và mục đích đối chiếu	12
Bảng 4.1:	Phiên bản phần mềm được cấu hình trong repo	13
Bảng 4.2:	Sức khỏe cụm sau pha phát lại	14
Bảng 4.3:	Kết quả phát lại theo giới hạn phát cấu hình.....	15
Bảng 4.4:	Kết quả hoán đổi MV công khai	17
Bảng 4.5:	Tổng hợp kết luận theo bằng chứng	19
Bảng 4.6:	Giới hạn bằng chứng và tác động tới kết luận	20

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Diễn giải
API	Application Programming Interface – giao diện lập trình ứng dụng
CSV	Comma-Separated Values – định dạng dữ liệu phân tách bằng dấu phẩy
DSR	Design Science Research – phương pháp nghiên cứu khoa học thiết kế
GitOps	Mô hình vận hành dùng Git làm nguồn trạng thái mong muốn
HA	High Availability – tính sẵn sàng cao
ITS	Intelligent Transportation System – hệ thống giao thông thông minh
JSONL	JSON Lines – định dạng mỗi dòng là một JSON độc lập
K3s	Bản phân phối Kubernetes gọn nhẹ của SUSE
MV	Materialized View – khung nhìn vật lý
PVC	PersistentVolumeClaim – yêu cầu cấp phát lưu trữ bền vững trong Kubernetes
S3	Simple Storage Service – giao diện lưu trữ đối tượng tương thích Amazon S3
SDG	Sustainable Development Goal – Mục tiêu Phát triển Bền vững
SQL	Structured Query Language – ngôn ngữ truy vấn có cấu trúc
TLC	Taxi and Limousine Commission – Ủy ban Taxi và Limousine Thành phố New York

TÓM TẮT BÁO CÁO

Nghiên cứu này xây dựng và đánh giá **Continuix**, một kiến trúc kho hồ dữ liệu thời gian thực (*real-time Data Lakehouse*) cho dữ liệu giao thông thông minh trên cụm K3s ba máy chủ. Hệ thống chuyển đổi dữ liệu NYC TLC Yellow Taxi từ Parquet sang JSONL, phát lại từng bản ghi thành sự kiện bằng Vector, chuyển luồng qua Redpanda, xử lý bằng SQL luồng trong RisingWave và ghi kết quả Apache Iceberg trên MinIO. Trạng thái triển khai được quản lý bằng Argo CD; chỉ số vận hành được lưu bằng VictoriaMetrics và trực quan hóa bằng Grafana. Hiện vật kỹ thuật được đánh giá qua bốn lượt chạy độc lập với giới hạn phát cấu hình lần lượt là 2, 1.000, 5.000 và 10.000 sự kiện mỗi giây. Ở cả bốn lượt, đầu ra Iceberg được tạo, Vector trở về trạng thái dừng sau phát lại và vòng lặp truy vấn không ghi nhận lỗi trong lúc hoán đổi khung nhìn vật lý công khai. Lượt benchmark-high ghi nhận 426.960 chuyển trước chuyển giao, 423.239 chuyển sau khi áp dụng cách lọc mới và thời lượng hoán đổi 0,215989 s. Kết quả chứng minh khả năng cập nhật cách tính truy vấn công khai mà không ghi nhận gián đoạn trong cửa sổ quan sát, đồng thời cho thấy quy trình thực nghiệm có thể lặp lại trên cụm tài nguyên giới hạn.

Từ khóa: kho hồ dữ liệu, xử lý luồng, khung nhìn vật lý, chuyển giao Blue/Green, RisingWave, Apache Iceberg, GitOps, K3s, hệ thống giao thông thông minh.

Chương 1. Mở đầu

1.1. Đặt vấn đề

Dữ liệu giao thông đô thị phát sinh liên tục: mỗi chuyến xe bổ sung thông tin về điểm đón, quãng đường, chi phí và nhu cầu theo khu vực. Trong hệ thống giao thông thông minh (*Intelligent Transportation System, ITS*), kết quả tổng hợp cần được cập nhật gần thời gian thực thay vì chỉ xuất hiện sau một đợt xử lý lô. Khi quy tắc chất lượng dữ liệu thay đổi, hệ thống cũng cần chuyển sang cách tính mới mà không làm gián đoạn truy vấn đang phục vụ.

Kho hồ dữ liệu (*Data Lakehouse*) kết hợp tính linh hoạt của hồ dữ liệu với khả năng quản trị bảng và phân tích có cấu trúc của kho dữ liệu [3]. Khi bổ sung xử lý luồng (*stream processing*), hệ thống có thể duy trì khung nhìn vật lý (*materialized view, MV*), tức bảng kết quả được cập nhật theo dữ liệu mới thay vì tính lại toàn bộ mỗi lần truy vấn. Bài báo Ursa [8] cho thấy nhu cầu thu hẹp khoảng cách giữa luồng sự kiện và lớp lưu trữ kho hồ dữ liệu. Từ động lực đó, đề tài tập trung vào một câu hỏi vận hành: trên cụm tài nguyên nhỏ, có thể phát lại dữ liệu, ghi đầu ra Iceberg, quan sát hệ thống và thay đổi cách tính MV công khai mà không ghi nhận lỗi truy vấn hay không?

1.2. Lý do chọn đề tài

Continux khảo sát câu hỏi trên bằng dữ liệu mở NYC TLC Yellow Taxi và các thành phần nguồn mở: K3s, Vector, Redpanda, RisingWave, MinIO, Apache Iceberg, Argo CD, VictoriaMetrics và Grafana. K3s giúp tái hiện các khái niệm cốt lõi của Kubernetes trên hạ tầng phòng thí nghiệm. Đề tài có liên hệ với Mục tiêu Phát triển Bền vững (*Sustainable Development Goal, SDG*) số 8, 9 và 11 thông qua nhu cầu hạ tầng số hỗ trợ dịch vụ di chuyển hiệu quả, đổi mới công nghệ và đô thị bền vững [14, 15, 16]. Liên hệ này là động lực ứng dụng, không phải phép đo định lượng tác động xã hội.

1.3. Mục tiêu nghiên cứu

- Thiết kế luồng dữ liệu từ NYC TLC đến đầu ra Apache Iceberg trên MinIO.
- Triển khai luồng trên cụm K3s ba máy chủ, quản lý trạng thái mong muốn bằng GitOps và quan sát bằng chỉ số vận hành.

- Chạy bốn lượt phát lại độc lập với giới hạn phát cấu hình từ thấp đến cao.
- Thực hiện chuyển giao Blue/Green (*Blue/Green cutover*) ở lớp MV công khai và kiểm tra lỗi truy vấn trong lúc hoán đổi tên.
- Chỉ kết luận từ dữ liệu có thể truy vết trong bộ bằng chứng đã lưu.

1.4. Đối tượng và phạm vi

Đối tượng xử lý là dữ liệu công khai NYC TLC Yellow Taxi tháng 2026-03 và bảng tra cứu Taxi Zone Lookup. Tập Parquet được chuyển thành JSONL gồm 3.952.451 dòng; bảng tra cứu có 265 dòng. Mỗi lượt chạy tạo trạng thái nền sạch, phát lại dữ liệu trong cửa sổ đo cố định, chốt đầu ra, thực hiện chuyển giao MV và dọn trạng thái sinh ra.

Bốn cấu hình Vector là smoke, benchmark-low, benchmark-medium và benchmark-high, tương ứng với giới hạn phát cấu hình 2, 1.000, 5.000 và 10.000 sự kiện mỗi giây. Đây là giới hạn đặt tại nguồn phát, không phải thông lượng đầu cuối đã đo. Báo cáo sử dụng số chuyển đã xuất hiện trong MV, đường tăng theo thời gian, trạng thái cụm và đầu ra Iceberg để đánh giá hiện tượng quan sát được.

Phạm vi chuyển giao giới hạn ở tên truy vấn công khai mv_zone_stats. Báo cáo không khẳng định rằng đầu ra ghi Iceberg tự động chuyển quan hệ phụ thuộc sang cách tính mới sau phép hoán đổi tên MV.

1.5. Câu hỏi nghiên cứu

1.6. Đóng góp và cấu trúc báo cáo

Đề tài đóng góp một hiện vật kỹ thuật có thể tái hiện, một quy trình thực nghiệm theo pha, bộ bằng chứng của bốn cấu hình phát và cách diễn giải thận trọng cho chuyển giao MV công khai. Chương 2 trình bày cơ sở lý thuyết. Chương 3 mô tả phương pháp và kiến trúc. Chương 4 trình bày kết quả. Chương 5 bàn luận giới hạn và kết luận. Phụ lục giữ SQL, cấu hình trích yếu và chỉ mục bằng chứng.

Bảng 1.1: Câu hỏi nghiên cứu và bằng chứng trả lời

ID	Câu hỏi	Bằng chứng	Kết luận trong phạm vi đo
RQ1	MV công khai có tiếp tục truy vấn được trong lúc hoán đổi cách tính?	Bốn vòng lặp truy vấn, bốn số đo thời lượng và chỉ số lỗi.	Có; cả bốn lượt ghi nhận 0 lỗi truy vấn.
RQ2	Luồng dữ liệu có tạo đầu ra kho hồ dữ liệu quan sát được?	Danh sách đối tượng Iceberg sau mỗi lượt phát lại.	Có; cả bốn lượt đều có Parquet và siêu dữ liệu Iceberg.
RQ3	Quy trình có lặp lại được trên cụm tài nguyên giới hạn?	Trạng thái nút, PVC, khối lượng công việc, Argo CD, Vector và kết quả dọn dẹp.	Có; bốn lượt độc lập đều hoàn tất và trở về trạng thái sạch.

Chương 2. Tổng quan và cơ sở lý thuyết

2.1. Kho hồ dữ liệu và Apache Iceberg

Kho dữ liệu (*data warehouse*) thường đưa dữ liệu vào các bảng được quản trị chặt để phục vụ phân tích ổn định. Ngược lại, hồ dữ liệu (*data lake*) lưu nhiều loại tệp trên hạ tầng chi phí thấp nhưng có thể thiếu lớp quản trị bảng thống nhất. Kho hồ dữ liệu (*Data Lakehouse*) kết hợp hai hướng: dữ liệu vẫn nằm ở định dạng mở, đồng thời được bổ sung siêu dữ liệu (*metadata*), lịch sử phiên bản và ngữ nghĩa giao dịch ACID (*Atomicity, Consistency, Isolation, Durability*) [3].

Lưu trữ đối tượng (*object storage*) truy cập dữ liệu theo khóa qua API HTTP thay vì đường dẫn tệp truyền thống. Trong Continux, MinIO cung cấp giao diện tương thích Amazon S3. Apache Iceberg là định dạng bảng (*table format*), không phải bộ máy truy vấn: đặc tả này mô tả tệp nào thuộc bảng, ảnh chụp trạng thái (*snapshot*) nào đang hiện hành và tệp kê khai (*manifest*) nào theo dõi thay đổi [1]. Đích ghi (*sink*) Iceberg có thể tạo tệp dữ liệu Parquet, tệp xóa theo giá trị khóa (*equality-delete*), tệp xóa theo vị trí dòng (*position-delete*) và siêu dữ liệu. Vì vậy, việc nhìn thấy các đối tượng trên MinIO chứng minh đường ghi đã hoạt động, không có nghĩa MinIO tự xử lý SQL.

2.2. Xử lý luồng, trạng thái và khung nhìn vật lý

Một sự kiện (*event*) trong đề tài là bản ghi chuyến taxi được phát lại dưới dạng JSON. Xử lý luồng cập nhật kết quả khi sự kiện đến, thay vì chờ toàn bộ tệp hoàn tất như xử lý lô. Khung nhìn vật lý (*materialized view, MV*) là kết quả truy vấn được hệ thống duy trì sẵn và cập nhật theo đầu vào. Với dữ liệu taxi, MV nhóm theo quận và khu vực, sau đó tính số chuyến, tổng cước và quãng đường trung bình.

Phép join với bảng tra cứu và phép tổng hợp theo vùng cần trạng thái (*state*), tức thông tin trung gian được giữ qua nhiều sự kiện. Điểm kiểm tra (*checkpoint*) là mốc lưu trạng thái có chủ đích để engine có cơ sở tiếp tục hoặc khôi phục xử lý sau gián đoạn. Continux dùng bucket `rw-checkpoint` trên MinIO cho trạng thái RisingWave.

2.3. Broker tương thích Kafka và RisingWave

Kafka định hình mô hình nhật ký phân tán cho xử lý luồng [7]. Chủ đề (*topic*) là luồng thông điệp có tên; phân vùng (*partition*) là phần chia phục vụ thứ tự cục bộ và xử lý song song; độ lệch (*offset*) là vị trí của thông điệp trong phân vùng. Phía phát

(*producer*) ghi thông điệp, còn phía nhận (*consumer*) đọc và ghi nhớ độ lệch đã xử lý.

Continex dùng Redpanda, broker tương thích Kafka API, để tách Vector khỏi RisingWave [11]. Topic `nyc-taxi-events` có ba phân vùng và một bản sao. Một bản sao nghĩa là đề tài không đánh giá khả năng chịu mất broker. RisingWave đọc topic, join với bảng tra cứu trên MinIO, duy trì MV và ghi kết quả ra Iceberg.

2.4. Chuyển giao Blue/Green

Blue/Green deployment là cách vận hành hai phiên bản song song để giảm rủi ro khi thay đổi hệ thống. Blue là phiên bản đang phục vụ truy vấn hiện tại; Green là phiên bản ứng viên đã được chuẩn bị để thay thế. Khi Green đủ sẵn sàng, thao tác chuyển giao (*cutover*) chuyển điểm truy cập công khai từ Blue sang Green. Trong Continex, điểm chuyển giao không phải là pod Kubernetes mà là định nghĩa MV: `mv_zone_stats` là tên công khai ổn định cho phía truy vấn, `mv_zone_stats_blue` giữ cách tính nền và `mv_zone_stats_green` áp dụng bộ lọc mới.

RisingWave hỗ trợ `ALTER MATERIALIZED VIEW ... SWAP WITH ...` để hoán đổi tên hai MV [13]. Tính nguyên tử (*atomic*) ở đây chỉ áp dụng cho thao tác đổi tên nhìn từ truy vấn trong bộ máy: phía truy vấn không thấy trạng thái tên trung gian. Điều này không tự động chứng minh mọi quan hệ phụ thuộc phía sau cũng đổi theo. Tài liệu RisingWave yêu cầu xem xét riêng các thành phần phía sau (*downstream*) như đích ghi khi thay đổi công việc xử lý luồng [12]. Vì vậy kết luận của đề tài chỉ áp dụng cho tên truy vấn công khai.

2.5. GitOps và khả năng quan sát

OpenGitOps xác định bốn nguyên tắc cho trạng thái mong muốn: khai báo, lưu phiên bản trong Git, tự động kéo về và liên tục điều hòa sai lệch (*reconcile*) [10]. Continex dùng Argo CD theo mẫu ứng dụng-gốc-quản-lý-các-ứng-dụng-con (*App-of-Apps*) [2]. Git giữ cấu hình dài hạn; việc scale Vector trong một lượt phát lại là thao tác đo có chủ đích.

Khả năng quan sát (*observability*) là khả năng suy luận trạng thái nội tại từ tín hiệu bên ngoài như chỉ số, nhật ký và trạng thái khối lượng công việc. VictoriaMetrics lưu chuỗi thời gian [17]; Grafana trực quan hóa chỉ số [5]; bộ xuất số đo (*exporter*) của Continex chuyển kết quả SQL và kết quả chuyển giao thành số đo. Bảng điều khiển (*dashboard*) hỗ trợ đọc hiện tượng, nhưng kết luận trong báo cáo luôn được đối chiếu với SQL và danh sách đối tượng MinIO.

2.6. Ursa và nghiên cứu liên quan

Ursa là bộ máy xử lý luồng tương thích Kafka và gắn trực tiếp với kho hồ dữ liệu, tập trung giảm chi phí sao chép và chi phí đầu nối (*connector*) [8]. Continuum không triển khai Ursa và không so sánh hiệu năng trực tiếp với Ursa. Đề tài khảo sát lớp vận hành phía trên: GitOps, quan sát hệ thống và thay đổi cách tính truy vấn công khai trên cụm nhỏ.

Flink hỗ trợ xử lý lô và xử lý luồng trong cùng một bộ máy [4]; Kafka Streams và ksqlDB đại diện cho hướng xử lý gắn với hệ sinh thái Kafka. RisingWave được chọn vì giao diện SQL, MV và cú pháp hoán đổi phù hợp với hiện vật. Đây là định vị khái niệm, không phải phép đo hiệu năng đối đầu.

Bảng 3.1: Nguồn dữ liệu và cấu hình phát lại

Thuộc tính	Giá trị
Dữ liệu chuyển xe	NYC TLC Yellow Taxi, tháng 2026-03
Định dạng nguồn / đích chuyển đổi	Parquet / JSONL
Số dòng JSONL xác nhận	3.952.451
Bảng tra cứu Taxi Zone Lookup	265 dòng
Giới hạn phát cấu hình	2, 1.000, 5.000, 10.000 sự kiện/s
Thời gian lấy mẫu khi phát lại	12 mẫu, cách nhau 5 s

Chương 3. Nghiên cứu thực nghiệm và kiến trúc hệ thống

3.1. Phương pháp nghiên cứu

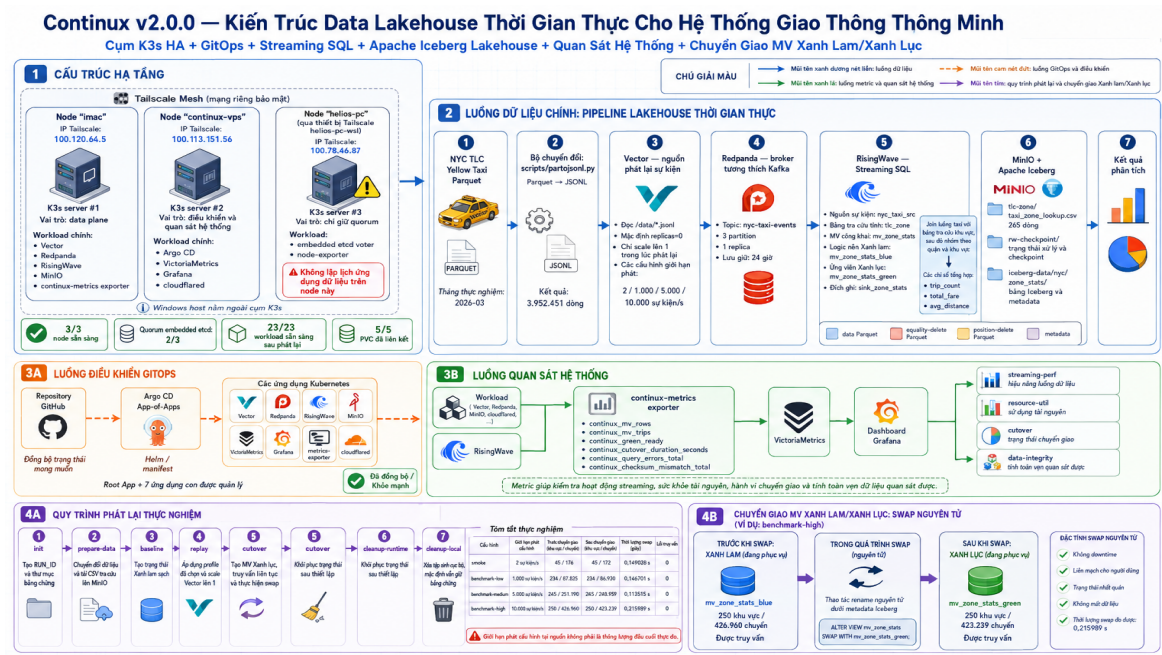
Nghiên cứu áp dụng phương pháp khoa học thiết kế (*Design Science Research*, DSR) [6]. Theo DSR, tri thức được tạo ra thông qua việc xây dựng và đánh giá một hiện vật kỹ thuật có chủ đích. Hiện vật của đề tài là toàn bộ kiến trúc Continux: cụm K3s, luồng dữ liệu từ Vector đến Iceberg, lớp quản lý triển khai Argo CD và lớp quan sát VictoriaMetrics/Grafana.

Đánh giá được thực hiện bằng bốn lượt chạy độc lập trên cùng bản sửa đổi Git ff19448. Mỗi lượt đi qua các pha: khởi tạo, chuẩn bị dữ liệu, tạo trạng thái nền sạch, phát lại, chuyển giao MV, xác minh cuối, dọn trạng thái chạy và dọn tệp cục bộ. Cách tổ chức này giúp mỗi cấu hình bắt đầu từ cùng một trạng thái và tránh trộn dữ liệu giữa các lượt.

3.2. Dữ liệu và cấu hình phát lại

Sở Taxi và Dịch vụ vận tải New York (NYC TLC) công bố dữ liệu chuyển xe công khai phục vụ phân tích [9]. Continux dùng Yellow Taxi tháng 2026-03. Tập lệnh `scripts/partojsonl.py` chuyển Parquet thành JSONL để Vector đọc tuần tự. Bảng Taxi Zone Lookup được tải lên MinIO để ánh xạ mã vị trí sang quận và khu vực.

Vector bổ sung `event_id` và `event_time`, rồi gửi từng sự kiện vào topic `nyc-taxi-events`. Tham số tốc độ chỉ giới hạn nguồn phát. Vì báo cáo không có phép đo số sự kiện đầu cuối trên mỗi giây, các cấu hình được dùng để so sánh xu hướng số chuyển đã xử lý trong cùng cửa sổ quan sát, không được diễn giải thành thông lượng thực tế.



Hình 3.1: Kiến trúc, luồng xử lý và quy trình chuyển giao của Continux 2.1.0.

Bảng 3.2: Cấu trúc cụm K3s thực nghiệm

Node	Tài nguyên ghi nhận	Vai trò
imac	Intel i5-8500, 6 core, 8 GB RAM, 200 GB SSD, Ubuntu 26.04	Máy chủ #1; Vector, Redpanda, RisingWave, MinIO, exporter
continux-vps	2 vCPU, 4 GB RAM, 80 GB SSD, Ubuntu 24.04	Máy chủ #2; Argo CD, VictoriaMetrics, Grafana
helios-pc	WSL Ubuntu 26.04, host Intel i5-12500H, 16 GB RAM	Máy chủ #3; quorum-only, node-exporter

3.3. Kiến trúc tổng thể

Luồng dữ liệu bắt đầu từ JSONL, đi qua Vector và Redpanda, sau đó được RisingWave xử lý bằng SQL luồng. RisingWave join sự kiện với bảng tra cứu, duy trì MV và ghi kết quả Iceberg lên MinIO. Luồng điều khiển dùng Argo CD để đồng bộ manifest từ Git. Luồng quan sát dùng exporter, VictoriaMetrics và Grafana để hiển thị trạng thái phát lại, tài nguyên, chuyển giao và tính toán vẹn quan sát được.

3.4. Cấu trúc triển khai

Ba node kết nối qua Tailscale. K3s dùng embedded etcd với quorum 2/3. Thuật ngữ sẵn sàng cao trong báo cáo chỉ áp dụng cho lớp điều khiển Kubernetes và etcd. Đường

dữ liệu không phải cấu hình sẵn sàng cao đầu cuối: Redpanda chạy một broker và topic dùng `replicationFactor=1`.

3.5. Đối tượng SQL và chuyển giao Blue/Green

Bảng 3.3: Đối tượng dữ liệu cốt lõi

Đối tượng	Đầu vào	Chức năng
tlc_zone	CSV trên MinIO	Tra cứu vị trí sang quận và khu vực
nyc_taxi_src	Topic Redpanda	Nguồn sự kiện JSON
mv_zone_stats_blue	Nguồn + bảng tra cứu	Cách tính nền Blue, giữ toàn bộ bản ghi hợp lệ ở trạng thái ban đầu
mv_zone_stats	MV Blue ban đầu	Tên công khai ổn định để phía truy vấn luôn gọi cùng một đối tượng
mv_zone_stats_green	Nguồn + tra cứu + bộ lọc	Ứng viên Green, dùng để kiểm tra logic mới trước khi chuyển giao
sink_zone_stats	MV công khai lúc tạo đích ghi	Đầu ra Iceberg kiểu cập nhật hoặc chèn

Cách tính Green bổ sung điều kiện loại các bản ghi có giá trị âm ở tiền cước hoặc quãng đường:

Listing 1: Điều kiện bổ sung trong cách tính Green

```
WHERE t.fare_amount >= 0
AND t.trip_distance >= 0
```

Khi MV Green đã có dữ liệu ổn định, vòng lặp truy vấn tiếp tục đọc tên công khai theo chu kỳ khoảng 0,5 s. Trong lúc vòng lặp này đang chạy, hệ thống thực hiện câu lệnh:

Listing 2: Câu lệnh hoán đổi tên MV

```
ALTER MATERIALIZED VIEW mv_zone_stats
SWAP WITH mv_zone_stats_green;
```

Phép hoán đổi giữ nguyên tên truy vấn công khai nhưng thay cách tính phía sau tên đó. Nhờ vậy, người dùng vẫn truy vấn cùng một tên MV, còn hệ thống chuyển từ logic Blue sang logic Green. Đích ghi Iceberg đã được tạo trước thao tác này; vì vậy

đầu ra Iceberg chứng minh đường ghi hoạt động trong pha phát lại, không chứng minh đích ghi tự chuyển sang cách tính Green.

3.6. Nguồn bằng chứng và khả năng truy vết

Bốn gói bằng chứng chính thức được sao chép từ iMac về Windows tại `evidence/finalize/20260531-four-profiles/`. Thư mục `runs/` chứa bốn `RUN_ID`: 20260531-172050, 20260531-173434, 20260531-174621 và 20260531-175701. Mỗi gói có 41 tệp; tổng cộng có 164 tệp văn bản hoặc JSON.

Tệp `SHA256SUMS.txt` chứa giá trị băm SHA-256 của từng tệp. Giá trị băm nguồn trên iMac và bản sao Windows khớp 164/164. Tệp `summary.csv` tổng hợp số liệu chính; tệp `sensitive-scan.txt` ghi kết quả rà chuỗi nhạy cảm. Các nhóm bằng chứng được dùng như sau:

Bảng 3.4: Nhóm tệp bằng chứng và mục đích đối chiếu

Nhóm	Nội dung đối chiếu
00-run-id.txt	Cấu hình và giới hạn phát cấu hình
04-mv-progress.txt	Đường tăng số vùng và số chuyển trong pha phát lại
04-mv-final.txt	Số vùng và số chuyển chốt sau phát lại
04-iceberg-final.txt	Đối tượng Parquet và siêu dữ liệu Iceberg
05-green-ready-samples.txt	Trạng thái MV Green trước chuyển giao
05-query-loop-during-cutover.txt	Truy vấn thành công trước và sau hoán đổi
05-duration-and-errors.txt	Thời lượng hoán đổi và số lỗi truy vấn
05-final-*	SQL cuối, Vector, sức khỏe cụm và Argo CD
06-*	Kết quả dọn trạng thái sinh ra

Chương 4. Kết quả thực nghiệm, đánh giá và bàn luận

4.1. Môi trường và phiên bản

Bảng 4.1: Phiên bản phần mềm được cấu hình trong repo

Thành phần	Phiên bản
Continux	2.1.0
K3s	v1.35.5+k3s1
Helm	v4.2.0
Argo CD	ứng dụng v3.4.3, gói Helm 9.5.17
Tailscale	v1.98.4
Redpanda	ứng dụng v26.1.9, gói Helm 26.1.4
RisingWave	ứng dụng v2.8.4, gói Helm 0.2.52
Vector	0.55.0-alpine
VictoriaMetrics	ứng dụng v1.144.0, gói Helm 0.81.0
Grafana	ảnh container 13.0.1-security-01, gói Helm 10.5.15
MinIO	gói Helm 5.4.0
cloudflared	2026.5.2

4.2. Trạng thái nền và khả năng lặp lại

Trước mỗi lượt phát lại, tập lệnh điều phối dừng Vector, xóa đối tượng SQL sinh bởi lượt trước, tạo lại chủ đề Redpanda, dọn tiền tố Iceberg và dựng lại trạng thái Blue sạch. Trạng thái Blue ở đây là logic truy vấn ban đầu, dùng làm mốc so sánh trước khi chuyển sang Green. Sau khi hoàn tất, tập lệnh tiếp tục dọn trạng thái chạy và tệp cục bộ. Vì vậy bốn cấu hình là bốn lượt độc lập, không phải bốn pha nối tiếp dùng chung dữ liệu.

Bảng 4.2: Sức khỏe cụm sau pha phát lại

Cấu hình	Nút Ready	PVC Bound	Khối lượng công việc sẵn sàng	RAM nút imac
smoke	3/3	5/5	23/23	47%
benchmark-low	3/3	5/5	23/23	50%
benchmark-medium	3/3	5/5	23/23	51%
benchmark-high	3/3	5/5	23/23	52%

Sau chuyển giao, tám ứng dụng Argo CD chính đều ở trạng thái Synced/Healthy; Vector trở về 0 desired trong cả bốn lượt. Ảnh chụp trạng thái cuối của ba cấu hình đầu ghi 23/23 khối lượng công việc sẵn sàng. Riêng ảnh chụp cuối của benchmark-high ghi 22/23: trong danh sách chi tiết, pod Grafana ở trạng thái Running nhưng chưa sẵn sàng 0/1. Đây là trạng thái chuyển tiếp sau phép đo; báo cáo không che giấu điểm này và không dùng nó để tuyên bố toàn bộ khối lượng công việc luôn sẵn sàng ở mọi thời điểm. Argo CD vẫn ghi nhận ứng dụng bằng điều khiển ở trạng thái Synced/Healthy.

4.3. Dữ liệu đầu vào có thể truy vết

Gói bằng chứng mới xác nhận tệp Yellow Taxi tháng 2026-03 được chuyển thành 3.952.451 dòng JSONL và bảng Taxi Zone Lookup có 265 dòng. Báo cáo không giữ các thống kê phân phối chi tiết của bản thảo cũ vì bốn gói chính thức không chứa phép quét dữ liệu thô tương ứng. Cách tính Green vẫn được mô tả đúng theo SQL đã chạy: loại các chuyến có fare_amount < 0 hoặc trip_distance < 0, tức các bản ghi có tiền cước hoặc quãng đường âm.

4.4. Kết quả phát lại theo bốn cấu hình

Bảng 4.3: Kết quả phát lại theo giới hạn phát cấu hình

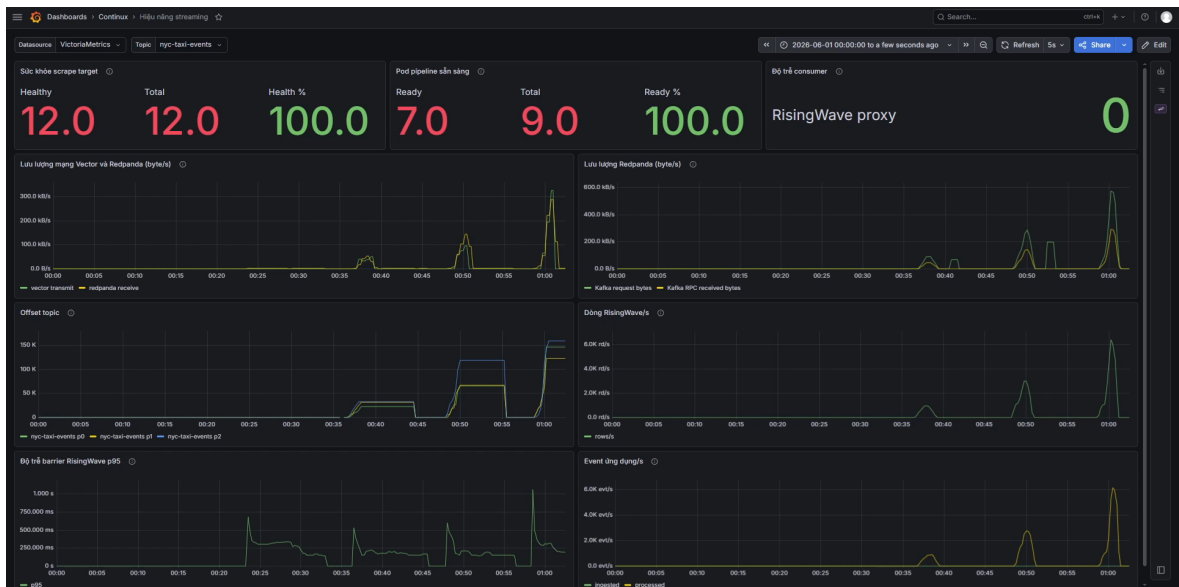
Cấu hình	Giới hạn cấu hình	Mẫu thứ 12	Chốt sau phát lại	Đối tượng Iceberg
smoke	2 sự kiện/s	39 vùng / 104 chuyển	45 / 176	Có
benchmark-low	1.000 sự kiện/s	223 / 51.825	234 / 87.825	Có
benchmark-medium	5.000 sự kiện/s	234 / 93.307	245 / 251.190	Có
benchmark-high	10.000 sự kiện/s	227 / 77.139	250 / 426.960	Có

Tập 04-mv-progress.txt của mỗi lượt gồm 12 mẫu cách nhau khoảng 5 s. Sau mẫu cuối, tập lệnh điều phối dừng Vector và chờ các sự kiện đã nhận được xử lý tiếp trước khi chốt 04-mv-final.txt. Vì vậy cột chốt sau phát lại cao hơn mẫu thứ 12. Kết quả cho thấy MV tăng trong cả bốn cấu hình và đường ghi Iceberg hoạt động ở cả bốn lượt.

Các số trên không phải thông lượng đầu cuối. Đặc biệt, không thể suy ra rằng cấu hình benchmark-high xử lý chính xác 10.000 sự kiện mỗi giây chỉ từ giá trị cấu hình. Việc số chuyển chốt tăng theo cấu hình là xu hướng quan sát được trong quy trình này; đo thông lượng và độ trễ đầu cuối cần bộ đo chuyên biệt.

Hình 4.1 ghi nhận các đợt lưu lượng phát lại, đường tăng vị trí đọc của ba phân vùng và các đợt xử lý của RisingWave. Tại thời điểm chụp, ô độ trễ phía tiêu thụ của bộ chuyển tiếp RisingWave hiển thị 0. Hình này là bằng chứng trực quan cho hoạt động của luồng xử lý trong cửa sổ quan sát; báo cáo không dùng đồ thị để suy diễn giới hạn phát cấu hình thành thông lượng đầu cuối thực đo.

Hình 4.2 ghi nhận 15/15 khối lượng công việc trong phạm vi bảng điều khiển tài nguyên đang sẵn sàng, đồng thời hiển thị diễn biến CPU, bộ nhớ và mức sử dụng PVC. Chỉ số 15/15 thuộc phạm vi truy vấn của bảng điều khiển tại thời điểm chụp; nó bổ sung cho, không thay thế, ảnh chụp sức khỏe cụm 23/23.



Hình 4.1: Bảng điều khiển hiệu năng xử lý luồng trong lượt benchmark-high.



Hình 4.2: Bảng điều khiển tài nguyên trong cửa sổ phát lại của lượt benchmark-high.

4.5. Kết quả chuyển giao Blue/Green

Bảng 4.4: Kết quả hoán đổi MV công khai

Cấu hình	Trước chuyển giao	MV công khai sau chuyển giao	Thời lượng	Lỗi truy vấn
smoke	45 vùng / 176 chuyển	45 / 172	0,149028 s	0
benchmark-low	234 / 87.825	234 / 86.930	0,146701 s	0
benchmark-medium	245 / 251.190	245 / 248.959	0,113515 s	0
benchmark-high	250 / 426.960	250 / 423.239	0,215989 s	0

Trong mỗi lượt, tệp 05-green-ready-samples.txt ghi nhận MV Green đã sẵn sàng trước chuyển giao. Tệp 05-query-loop-during-cutover.txt cho thấy truy vấn công khai vẫn thành công trước và sau thao tác hoán đổi. Tổng số chuyển giảm sau chuyển giao vì logic Green loại các bản ghi có giá trị âm; số vùng không đổi trong từng lượt. Cả bốn vòng lặp đều ghi nhận 0 lỗi.

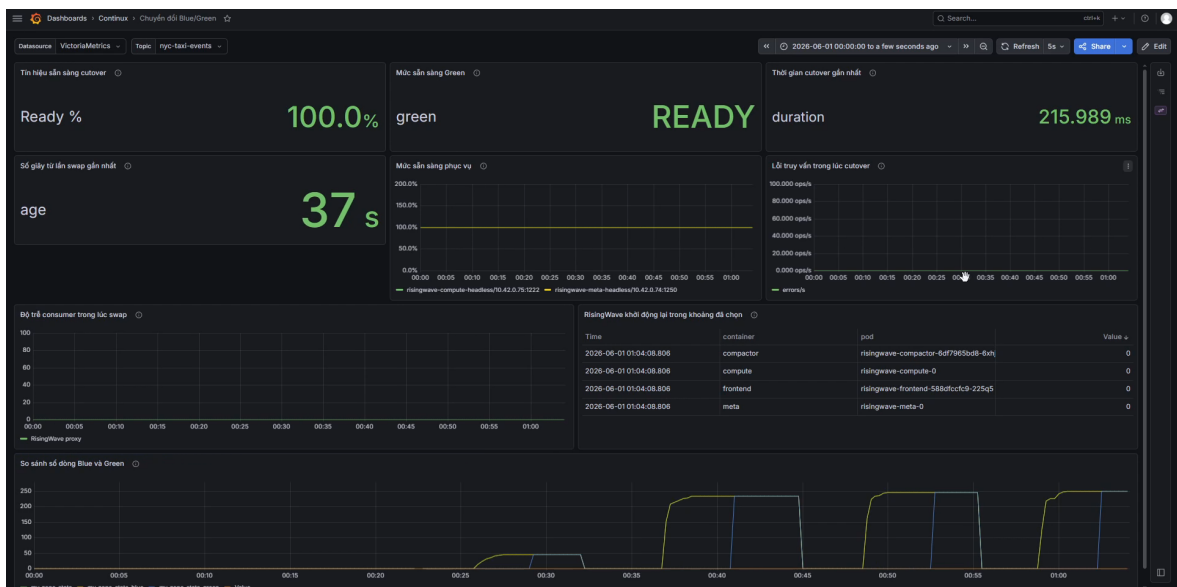
Thời lượng trong Bảng 4.4 được tập lệnh chạy đo quanh câu lệnh ALTER MATERIALIZED VIEW ... SWAP WITH Đây là chỉ báo vận hành của thao tác SQL trên cụm thử nghiệm, không phải cam kết mức dịch vụ và không đại diện cho mọi mẫu truy vấn hay mọi tình huống lỗi mạng.

Hình 4.3 đối chiếu trực quan kết quả cấu hình cao: MV Green ở trạng thái READY, mức sẵn sàng 100%, thời lượng chuyển giao gần nhất 215,989 ms và đường lỗi truy vấn bằng 0. Các giá trị này khớp với số liệu 0,215989 s và 0 lỗi trong gói bằng chứng.

Hình 4.4 ghi nhận MV công khai có 250 dòng vùng và ô checksum mismatch (không khớp tổng kiểm) bằng 1. Đây là kết quả dự kiến khi so MV công khai đang dùng logic Green với MV còn giữ logic Blue: hai logic lọc dữ liệu khác nhau nên tổng kiểm có thể khác nhau. Kết quả này không phải dấu hiệu mất dữ liệu.

4.6. Đầu ra Iceberg và trạng thái sau dọn dẹp

Danh sách đối tượng sau phát lại của cả bốn lượt đều có tệp dữ liệu Parquet, tệp equality-delete Parquet (xóa theo giá trị bằng nhau), tệp position-delete Parquet (xóa theo vị trí) và siêu dữ liệu Iceberg. Điều này chứng minh đích ghi đã lưu



Hình 4.3: Bảng điều khiển chuyển giao sau hoán đổi MV công khai trong lượt benchmark-high.



Hình 4.4: Bảng điều khiển tính toán vận quan sát được sau chuyển giao trong lượt benchmark-high.

đầu ra vào kho hồ dữ liệu trong pha phát lại. Sau khi xác minh cuối, tập lệnh điều phối dọn đối tượng SQL, đầu ra Iceberg, CSV tra cứu và trạng thái phát lại. Tập 06-post-setup-sql-count.txt của từng gói bằng chứng ghi demo_sql_objects = 0; thư mục làm việc trên iMac không còn data/raw/, .venv/, tệp trạng thái hiện hành hoặc CSV sinh tạm.

4.7. Tổng hợp đánh giá

Bảng 4.5: Tổng hợp kết luận theo bằng chứng

Nhóm	Bằng chứng ghi nhận	Kết luận
Chuyển giao lớp truy vấn	Bốn thời lượng, bốn vòng lặp truy vấn, lỗi đều 0	Không ghi nhận gián đoạn truy vấn MV công khai
Đầu ra kho hồ dữ liệu	Đối tượng Iceberg trong cả bốn gói bằng chứng	Đích ghi đã lưu đầu ra trong pha phát lại
Xu hướng theo cấu hình	MV tăng trong bốn cấu hình	Tải cấu hình cao hơn tạo nhiều kết quả hơn trong cửa sổ chạy; chưa phải phép đo thông lượng
Khả năng lặp lại	Bốn trạng thái nền sạch, bốn lượt dọn SQL về 0 đối tượng	Quy trình có thể chạy lặp lại
Sức khỏe cụm	Nút, PVC, khối lượng công việc, Argo CD và Vector	Cụm đủ vận hành kịch bản; ảnh chụp cuối của cấu hình cao có một trạng thái Grafana chuyển tiếp

4.8. Thảo luận kết quả

RQ1 được trả lời tích cực trong phạm vi quan sát: cả bốn lượt hoán đổi MV công khai đều hoàn tất, vòng lặp truy vấn chuyển từ logic Blue sang logic Green và không ghi nhận lỗi. Nói cách khác, phía truy vấn vẫn gọi cùng một tên MV công khai, còn hệ thống thay logic tính toán phía sau tên đó. Các thời lượng từ 0,113515 s đến 0,215989 s là chỉ báo vận hành cho thao tác SQL nhỏ trên cụm phòng thí nghiệm. Chúng không phải cam kết mức dịch vụ và không chứng minh mọi phía truy vấn hay mọi tình huống lỗi mạng đều không bị ảnh hưởng.

RQ2 được trả lời bằng danh sách đối tượng trên MinIO: mỗi lượt phát lại đều tạo tệp dữ liệu Parquet, tệp equality-delete Parquet, tệp position-delete Parquet và siêu dữ liệu Iceberg. Tuy nhiên, đích ghi đã được tạo trước phép hoán đổi tên MV. Bằng chứng chỉ chứng minh đường ghi kho hồ dữ liệu hoạt động trong pha phát lại; nghiên cứu không khẳng định đích ghi tự chuyển phần phụ thuộc phía sau sang logic Green sau chuyển giao.

RQ3 được trả lời bằng bốn chu kỳ trạng thái nền, phát lại, xác minh và dọn dẹp. Cả bốn lượt đều bắt đầu từ trạng thái sạch, kết thúc với Vector ở θ desired và dọn đối tượng SQL về 0. Ảnh chụp trạng thái sau phát lại luôn ghi 3/3 nút Ready, 5/5 PVC Bound và 23/23 khối lượng công việc sẵn sàng. Ảnh chụp cuối của cấu hình cao ghi Grafana tạm thời chưa sẵn sàng; đây là một điểm cần ghi nhận, đồng thời cho thấy đánh giá dài hạn cần thêm phép chạy bền vững thay vì chỉ dựa trên một cửa sổ ngắn.

So với Ursa, Continux không đề xuất bộ máy mới. Giá trị bổ sung của đề tài là ghép luồng kho hồ dữ liệu với GitOps, quan sát vận hành, quy trình dọn sạch có thể lặp lại và thay đổi cách tính truy vấn công khai trên cụm nhỏ.

4.9. Hạn chế

Bảng 4.6: Giới hạn bằng chứng và tác động tới kết luận

Giới hạn	Tác động
Các cấu hình đặt giới hạn phát tại Vector nhưng không đo số sự kiện đầu cuối mỗi giây	Không tuyên bố thông lượng hoặc độ trễ đầu cuối
Pha phát lại lấy mẫu trong cửa sổ cố định và còn giai đoạn xử lý tiếp sau khi dừng Vector	Chỉ so sánh xu hướng số chuyển chốt, không suy rộng thành công suất tối đa
Đích ghi Iceberg được tạo trước phép hoán đổi tên MV	Không kết luận đích ghi đã chuyển sang cách tính Green
Redpanda chạy một broker và topic dùng replicationFactor=1	Không đánh giá khả năng chịu lỗi của đường dữ liệu khi mất broker
Ảnh chụp cuối cấu hình high có Grafana Running nhưng chưa sẵn sàng 0/1	Cần phép chạy bền vững dài hơn để đánh giá ổn định theo thời gian
Bảng chứng chính thức xác nhận số dòng JSONL và lookup nhưng không chứa phép quét phân phối dữ liệu thô	Không giữ các kết luận chi tiết về chất lượng dữ liệu của bản thảo cũ

Chương 5. Kết luận

Continux đã xây dựng một hiện vật kho hồ dữ liệu thời gian thực trên K3s gồm nguồn phát Vector, broker Redpanda, SQL luồng RisingWave, đầu ra Apache Iceberg trên

MinIO, quản lý GitOps và lớp quan sát. Báo cáo cuối dựa trên bốn lượt chạy độc lập với giới hạn phát cầu hình 2, 1.000, 5.000 và 10.000 sự kiện mỗi giây.

Trong cả bốn lượt, MV tăng trong pha phát lại, đích ghi tạo đối tượng Iceberg, Vector trở về trạng thái dừng và vòng lặp truy vấn ghi nhận 0 lỗi khi tên MV công khai chuyển từ logic Blue sang logic Green. Lượt benchmark-high chốt 426.960 chuyển trước chuyển giao và 423.239 chuyển sau chuyển giao; thời lượng hoán đổi do tập lệnh chạy ghi là 0,215989 s.

Đóng góp thực nghiệm chính là chứng minh quy trình có thể lặp lại và cập nhật cách tính ở lớp truy vấn công khai mà không ghi nhận gián đoạn trong cửa sổ đo. Kết luận không mở rộng thành tuyên bố thông lượng đầu cuối, khả năng chịu lỗi của broker hoặc việc đích ghi Iceberg tự chuyển sang cách tính mới.

Chương 6. Hướng phát triển

- Bổ sung bộ đo thông lượng và độ trễ đầu cuối dựa trên mốc thời gian sự kiện, số sự kiện phía phát chấp nhận và số sự kiện phía nhận xử lý.
- Chạy phép đo bền vững dài hơn, kiểm tra phục hồi khi pod hoặc broker bị gián đoạn và theo dõi mức sử dụng tài nguyên theo thời gian.
- Thiết kế chuyển giao bao gồm cả đích ghi phía sau, chẳng hạn tạo đích ghi ứng viên gắn với MV Green rồi chuyển phía nhận theo phiên bản để luồng lưu trữ cũng đi theo logic mới.
- Tự động hóa điều kiện chuyển giao bằng quy trình có điều kiện bảo vệ và xuất manifest bằng chứng sau mỗi lượt.

Các nội dung trên là định hướng cho phiên bản kế tiếp, không phải kết quả của bốn lượt thực nghiệm đã báo cáo.

TÀI LIỆU THAM KHẢO

Tài liệu tiếng Việt

Không có tài liệu tiếng Việt được trích dẫn trong báo cáo.

Tài liệu tiếng Anh

- [1] Apache Software Foundation. “Apache iceberg documentation,” Accessed: May 24, 2026. [Online]. Available: <https://iceberg.apache.org/docs/latest/>
- [2] Argo Project. “Argo cd documentation,” Accessed: May 24, 2026. [Online]. Available: <https://argo-cd.readthedocs.io/>
- [3] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, “Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics,” in *Proceedings of the 11th Conference on Innovative Data Systems Research (CIDR)*, 2021. [Online]. Available: <https://www.vldb.org/cidrdb/2021/lakehouse-a-new-generation-of-open-platforms-that-unify-data-warehousing-and-advanced-analytics.html>
- [4] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, “Apache flink: Stream and batch processing in a single engine,” *IEEE Data Engineering Bulletin*, vol. 38, no. 4, pp. 28–38, 2015. [Online]. Available: <https://sites.computer.org/debull/A15dec/p28.pdf>
- [5] Grafana Labs. “Grafana documentation,” Accessed: May 24, 2026. [Online]. Available: <https://grafana.com/docs/grafana/latest/>
- [6] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. doi: 10.2307/25148625
- [7] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the NetDB Workshop*, 2011. [Online]. Available: <https://cwiki.apache.org/confluence/download/attachments/27822226/Kafka-netdb-06-2011.pdf>
- [8] M. Merli, S. Guo, P. Li, H. Chen, and N. Lu, “Ursa: A lakehouse-native data streaming engine for kafka,” *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 5184–5196, 2025. doi: 10.14778/3750601.3750636 [Online]. Available: <https://www.vldb.org/pvldb/vol18/p5184-guo.pdf>
- [9] New York City Taxi and Limousine Commission. “Tlc trip record data,” Accessed: May 24, 2026. [Online]. Available: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- [10] OpenGitOps Working Group. “Opengitops principles,” Accessed: May 24, 2026. [Online]. Available: <https://opengitops.dev/>
- [11] Redpanda Data. “Redpanda documentation,” Accessed: May 24, 2026. [Online]. Available: <https://docs.redpanda.com/>

- [12] RisingWave Labs. “Alter a streaming job,” Accessed: May 24, 2026. [Online]. Available: <https://docs.risingwave.com/operate/alter-streaming>
- [13] RisingWave Labs. “Alter materialized view: Swap with,” Accessed: May 24, 2026. [Online]. Available: <https://docs.risingwave.com/sql/commands/sql-alter-materialized-view>
- [14] United Nations. “Goal 11: Sustainable cities and communities,” Accessed: May 24, 2026. [Online]. Available: <https://sdgs.un.org/goals/goal11>
- [15] United Nations. “Goal 8: Decent work and economic growth,” Accessed: May 24, 2026. [Online]. Available: <https://sdgs.un.org/goals/goal8>
- [16] United Nations. “Goal 9: Industry, innovation and infrastructure,” Accessed: May 24, 2026. [Online]. Available: <https://sdgs.un.org/goals/goal9>
- [17] VictoriaMetrics. “Victoriametrics documentation,” Accessed: May 24, 2026. [Online]. Available: <https://docs.victoriametrics.com/>

Phụ lục A. SQL cốt lõi

A.1. Nguồn sự kiện, bảng tra cứu và MV nền

```
CREATE SOURCE IF NOT EXISTS nyc_taxi_src (  
    event_id VARCHAR,  
    event_time TIMESTAMPTZ,  
    pickup_time TIMESTAMPTZ,  
    pu_location_id INT,  
    do_location_id INT,  
    fare_amount DOUBLE PRECISION,  
    trip_distance DOUBLE PRECISION  
) WITH (  
    connector = 'kafka',  
    topic = 'nyc-taxi-events',  
    properties.bootstrap.server = 'redpanda.redpanda:9093',  
    scan.startup.mode = 'earliest'  
) FORMAT PLAIN ENCODE JSON;  
  
CREATE TABLE IF NOT EXISTS tlc_zone (  
    location_id INT PRIMARY KEY,  
    borough VARCHAR,  
    zone VARCHAR,  
    service_zone VARCHAR  
) WITH (  
    connector = 's3',  
    s3.bucket_name = 'tlc-zone',  
    s3.endpoint_url = 'http://minio.minio.svc.cluster.local:9000',  
    enable_config_load = 'true',  
    match_pattern = 'taxi_zone_lookup.csv'  
) FORMAT PLAIN ENCODE CSV (delimiter = ',', without_header = false);  
  
CREATE MATERIALIZED VIEW IF NOT EXISTS mv_zone_stats_blue AS  
SELECT z.borough, z.zone, COUNT(*) AS trip_count,  
       SUM(t.fare_amount) AS total_fare,  
       AVG(t.trip_distance) AS avg_distance  
FROM nyc_taxi_src t  
JOIN tlc_zone z ON t.pu_location_id = z.location_id  
GROUP BY z.borough, z.zone;
```

```
CREATE MATERIALIZED VIEW IF NOT EXISTS mv_zone_stats AS
SELECT * FROM mv_zone_stats_blue;
```

A.2. MV Green và thao tác hoán đổi

```
CREATE MATERIALIZED VIEW IF NOT EXISTS mv_zone_stats_green AS
SELECT z.borough, z.zone, COUNT(*) AS trip_count,
       SUM(t.fare_amount) AS total_fare,
       AVG(t.trip_distance) AS avg_distance
FROM nyc_taxi_src t
JOIN tlc_zone z ON t.pu_location_id = z.location_id
WHERE t.fare_amount >= 0
      AND t.trip_distance >= 0
GROUP BY z.borough, z.zone;

ALTER MATERIALIZED VIEW mv_zone_stats
SWAP WITH mv_zone_stats_green;
```

A.3. Đích ghi Iceberg

```
CREATE SINK IF NOT EXISTS sink_zone_stats FROM mv_zone_stats
WITH (
    connector = 'iceberg',
    type = 'upsert',
    primary_key = 'borough,zone',
    catalog.type = 'storage',
    warehouse.path = 's3://iceberg-data/',
    s3.endpoint = 'http://minio.minio.svc.cluster.local:9000',
    enable_config_load = 'true',
    database.name = 'nyc',
    create_table_if_not_exists = 'true',
    table.name = 'zone_stats'
);
```

Phụ lục B. Cấu hình vận hành trích yếu

```
# Redpanda topic
topics:
  - name: nyc-taxi-events
    partitions: 3
```

```

replicationFactor: 1
config:
  retention.ms: "86400000"

# Vector profiles: configured source-side limits
smoke:          2 events/s
benchmark-low:  1000 events/s
benchmark-medium: 5000 events/s
benchmark-high: 10000 events/s

# RisingWave state store
stateStore:
  s3:
    enabled: true
    endpoint: http://minio.minio.svc.cluster.local:9000
    bucket: rw-checkpoint

```

Phụ lục C. Chỉ mục bằng chứng

Bốn gói bằng chứng chính thức nằm ngoài Git tại:

```

evidence/finalize/20260531-four-profiles/
  SHA256SUMS.txt
  summary.csv
  sensitive-scan.txt
  runs/
    20260531-172050/ # smoke
    20260531-173434/ # benchmark-low
    20260531-174621/ # benchmark-medium
    20260531-175701/ # benchmark-high

```

Mỗi gói có 41 tệp. Các tệp cần ưu tiên khi kiểm tra lại số liệu là:

- 00-run-id.txt: cấu hình và giới hạn phát cấu hình.
- 04-mv-progress.txt, 04-mv-final.txt: tiến trình và số liệu chốt sau phát lại.
- 04-iceberg-final.txt: đối tượng Iceberg.
- 05-green-ready-samples.txt: MV Green đã sẵn sàng trước chuyển giao.
- 05-query-loop-during-cutover.txt, 05-duration-and-errors.txt: truy vấn trong lúc hoán đổi, thời lượng và lỗi.

- 05-final-*: SQL, Vector, sức khỏe cụm và Argo CD ở bước xác minh cuối.
- 06-*: kết quả dọn trạng thái sinh ra.

Phụ lục D. Danh sách hình minh chứng

PDF tích hợp năm hình minh chứng được dựng hoặc chụp từ hệ thống Continux 2.1.0 và bốn lượt đo chính thức:

1. Sơ đồ kiến trúc, luồng xử lý và quy trình chuyển giao Continux 2.1.0.
2. Bảng điều khiển streaming-perf của lượt benchmark-high, thể hiện các đợt lưu lượng, offset và xử lý luồng.
3. Bảng điều khiển resource-util trong cửa sổ phát lại benchmark-high, thể hiện mức sẵn sàng, CPU, bộ nhớ và PVC.
4. Bảng điều khiển cutover sau hoán đổi, thể hiện trạng thái READY, thời lượng và lỗi truy vấn.
5. Bảng điều khiển data-integrity sau chuyển giao; giá trị không khớp bằng 1 là kết quả dự kiến khi so hai cách tính khác nhau.